BCSE102L
Structured and Object Oriented Programming

C PROGRAMMING LANGUAGE – MODULE-1

By,
Dr. S. Vinila Jinny,
ASP/SCOPE

SYLLABUS

Module:1	C Programming Fundamentals	2 hours
Variables - Reserved words – Data Types – Operators – Operator Precedence - Expressions - Type Conversions - I/O statements - Branching and Looping: if, if-else, nested if, if-else ladder, switch statement, goto statement - Loops: for, while and do…while – break and continue statements.		
Module:2	**Arrays and Functions**	**4 hours**
Arrays: One Dimensional array - Two-Dimensional Array – Strings and its operations. User Defined Functions: Declaration – Definition – call by value and call by reference - Types of Functions - Recursive functions - Storage Classes - Scope, Visibility and Lifetime of Variables.		
Module:3	**Pointers**	**4 hours**
Declaration and Access of Pointer Variables, Pointer arithmetic – Dynamic memory allocation – Pointers and arrays - Pointers and functions.		
Module:4	**Structure and Union**	**2 hours**
Declaration, Initialization, Access of Structure Variables - Arrays of Structure - Arrays within Structure - Structure within Structures - Structures and Functions – Pointers to Structure -		

WHAT IS C LANGUAGE:-

- C is mother language of all programming language.
- It is a popular computer programming language.
- It is procedure-oriented programming language.
- It is also called mid level programming language.

HISTORY OF C LANGUAGE:-

- C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T(American Telephone & Telegraph), located in U.S.A.
- Dennis Ritchie is known as founder of c language.
- It was developed to be used in UNIX Operating system.
- It inherits many features of previous languages such as B and BPCL.

HISTORY OF C PROGRAMMING

Language	year	Developed By
ALGOL	1960	International Group
BPCL	1967	Martin Richards
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie
K & R C	1978	Kernighan & Dennis Ritchie
ANSI C	1989	ANSI Committee
ANSI/ISO C	1990	ISO Committee
C99	1999	Standardization Committee

FEATURES OF C LANGUAGE:-

There are many features of c language are given below.

1) Machine Independent or Portable
2) Mid-level programming language
3) structured programming language
4) Rich Library
5) Memory Management
6) Fast Speed
7) Pointers
8) Recursion
9) Extensible

FIRST PROGRAM OF C LANGUAGE:-

```
#include <stdio.h>
#include <conio.h>
void main(){
printf("VIT-SCOPE");
 }
```

DESCRIBE THE C PROGRAM :-

- **#include <stdio.h>** includes the **standard input output** library functions. The printf() function is defined in stdio.h .
- **#include <conio.h>** includes the **console input output** library functions. The getch() function is defined in conio.h file.
- **void main()** The **main() function is the entry point of every program** in c language. The void keyword specifies that it returns no value.
- **printf()** The printf() function is **used to print data** on the console.
- **getch()** The getch() function **asks for a single character**. Until you press any key, it blocks the screen.

OUTPUT OF PROGRAM IS:-

```
VIT-SCOPE
```

INPUT OUTPUT FUNCTION:-

There are two I/O function of c language.

1) First is printf()
2) Second is scanf()

- printf() function is used for output.
- It prints the given statement to the console.
- Syntax of printf() is given below:

printf("format string",arguments_list);

- Format string can be %d(integer), %c(character), %s(string), %f(float) etc.

INPUT/ OUTPUT FUNCTION

- scanf() Function: is used for input.
- It reads the input data from console.
- Syntax:

```
scanf("format string",argument_list);
void main()
{
int x;
scanf("%d",&x);
printf("X=%d",x);
}
```

DATA TYPES IN C LANGUAGE:-

- There are four types of data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

STORAGE SIZE & VALUE RANGE

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295

STORAGE SIZE & VALUE RANGE

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

KEYWORDS IN C LANGUAGE:-

- A keyword is a **reserved word**. We cannot use it as a variable name, constant name etc.
- There are 32 keywords in C language as given below:

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

OPERATORS IN C

- An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

Precedence & Associativity of Operators in C

- The precedence of operator species that which operator will be evaluated first and next. The associativity specifies the operator direction to be evaluated; it may be left to right or right to left.
- E.g:

int value=10+20*10;

TYPES OF OPERATOR

- Based on number of operands
  - Unary
  - Binary
  - Ternary
- Based on type of operation
  - Refer next slide

OPERATORS IN C LANGUAGE:-

- There are following types of operators to perform different types of operations in C language.

1) Arithmetic Operators
2) Relational Operators
3) Logical Operators
4) Bitwise Operators
5) Assignment Operator
6) Ternary or Conditional Operators
7) Misc Operator

ARITHMETIC OPERATORS

Operator	Description	Example A=10, B=20
+	Adds two operands.	A + B = 30
−	Subtracts second operand from the first.	A − B = -10
*	Multiplies both operands.	A * B = 200
/	Divides numerator by de-numerator.	B / A = 2
%	Modulus Operator and remainder of after an integer division.	B % A = 0
++	Increment operator increases the integer value by one.	A++ = 11
--	Decrement operator decreases the integer value by one.	A-- = 9

RELATIONAL OPERATORS

Operator	Description	Example A=10 B=20
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.

LOGICAL OPERATORS

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
||	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A || B) is true.
!	Logical NOT : It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true.

BITWISE OPERATORS

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) = 12, i.e., 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) = 61, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) = 49, i.e., 0011 0001
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A) = ~(60), i.e,. -0111101
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 = 240 i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 = 15 i.e., 0000 1111

ASSIGNMENT OPERATOR

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	C = A + B will assign the value of A + B to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	C %= A is equivalent to C = C % A

ASSIGNMENT OPERATOR

Operator	Description	Example
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
|=	Bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

TERNARY OR CONDITIONAL OPERATOR

- **Syntax:**
  The conditional operator is of the form

variable = Expression1 ? Expression2 : Expression3

- *Example:*

```
#include <stdio.h>
 int main()
{
    int m = 5, n = 4;
     (m > n) ? printf("m is greater”): printf("n is greater);
     return 0;
}
```

MISC OPERATOR

Operator	Description	Example
sizeof()	Returns the size of a variable.	sizeof(a), where a is integer, will return 4.
&	Returns the address of a variable.	&a; returns the actual address of the variable.
*	Pointer to a variable.	*a;

PRECEDENCE & ASSOCIATIVITY

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

TYPE CONVERSION

- **If a data type is automatically converted into another data type at compile time is known as type conversion.**
- The conversion is performed by the compiler if both data types are compatible with each other.
- Remember that the destination data type should not be smaller than the source type.
- It is also known as **widening** conversion of the data type.
- Eg: **int -> float**
- These are data types compatible with each other because their types are numeric, and the size of int is 2 bytes which is smaller than float data type. Hence, the compiler automatically converts the data types without losing or truncating the values.

TYPE CASTING

- Typecasting allows us to convert one data type into other. In C language, we use cast operator for typecasting which is denoted by (type).
- Syntax:

(type)value;

TYPE CASTING-EXAMPLE

```
#include <stdio.h>
void main()
 {
   int sum = 17, count = 5;
   double mean;
   mean = (double) sum / count;
   printf("Value of mean : %f\n", mean );
}
```

CONTROL STATEMENT IN C LANGUAGE:-

1) if-else
2) switch
3) loops
4) do-while loop
5) while loop
6) for loop
7) break
8) Continue
9) goto

C IF ELSE STATEMENT:-

- There are many ways to use if statement in C language:

1) If statement
2) If-else statement
3) If else-if ladder
4) Nested if

IF STATEMENT:-

- if statement is used to execute the code if condition is true.
- syntax:-

```
if(expression){
//code to be execute
}
```

IF STATEMENT:-

```
#include<stdio.h>
int main(){
int number=0;
printf("Enter a number:");
scanf("%d",&number);
if(number%2==0){
printf("%d is even number",number);
}
return 0;
}
```

IF ELSE STATEMENT:-

- The if-else statement is used to execute the code if condition is true or false.
- Syntax:

```
if(expression)
{
  //code to be executed if condition is true
}
else
{
  //code to be executed if condition is false
}
```

IF ELSE STATEMENT:-

```
#include <stdio.h>
int main()
{
    int age;
    printf("Enter your age?");
    scanf("%d",&age);
    if(age>=18)
    {
        printf("You are eligible to vote...");
    }
    else
    {
        printf("Sorry ... you can't vote");
    }
}
```

IF ELSE-IF LADDER STATEMENT:-

Syntax:

```
if(condition1)
{
//code to be executed if condition1 is true
}
else if(condition2)
{
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

IF ELSE-IF LADDER STATEMENT:-

```
#include <stdio.h>
int main()
{
    int marks;
    printf("Enter your marks?");
    scanf("%d",&marks);
    if(marks > 85 && marks <= 100)
    {
printf("Congrats ! you scored grade A
...");
    }
    else if (marks > 60 && marks <= 85
)
    {
        printf("You scored grade B + ...")
;
    }
    else if (marks > 40 && marks <= 60)
    {
        printf("You scored grade B ...");
    }
    else if (marks > 30 && marks <= 40)
    {
        printf("You scored grade C ...");
    }
    else
    {
        printf("Sorry you are fail ...");
    }
}
```

C SWITCH STATEMENT:-

- **Syntax:**

```
switch(expression)
{
case value1:
 //code to be executed;
 break;  //optional
case value2:
 //code to be executed;
 break;  //optional
.....
default:
code to be executed if all cases are not matched;
}
```

C SWITCH STATEMENT:-

```
#include <stdio.h>
int main()
{
    int x = 10, y = 5;
    switch(x>y && x+y>0)
    {
        case 1:
        printf("hi");
        break;
        case 0:
        printf("bye");
        break;
        default:
        printf(" Hello bye ");
    }

}
```

LOOPS IN C LANGUAGE:-

- Loops are used to execute a block of code or a part of program of the program several times.

Types of loops in C language:-

- There are 3 types of loops in c language.

1) do while
2) while
3) for

DO-WHILE LOOP IN C:-

- It is better if you have to execute the code at least once.
- Syntax:-

```
do{
//code to be executed
}while(condition);
```

DO-WHILE LOOP IN C:-

```
#include<stdio.h>
int main(){
int i=1;
do{
printf("%d \n",i);
i++;
}while(i<=10);
return 0;
}
```

WHILE LOOP IN C LANGUAGE:-

- It is better if number of iteration is not known by the user.
- Syntax:-

```
while(condition){
//code to be executed
}
```

WHILE LOOP IN C LANGUAGE:-

```
#include<stdio.h>
int main(){
int i=1;
while(i<=10){
printf("%d \n",i);
i++;
}
return 0;
}
```

FOR LOOP IN C LANGUAGE:-

- It is good if number of iteration is known by the user.
- Syntax:-

```
for(initialization;condition;incr/decr){
//code to be executed
}
```

FOR LOOP IN C LANGUAGE:-

```
#include<stdio.h>
int main(){
int i=1,number=0;
printf("Enter a number: ");
scanf("%d",&number);
for(i=1;i<=10;i++){
printf("%d \n",(number*i));
}
return 0;
}
```

C BREAK STATEMENT:-

- it is used to break the execution of loop (while, do while and for) and switch case.
- Syntax:-

jump-statement;

break;

C BREAK STATEMENT:-

```
#include<stdio.h>
void main ()
{
    int i = 0;
    while(1)
    {
        printf("%d  ",i);
        i++;
        if(i == 10)
        break;
    }
    printf("came out of while loop");
}
```

CONTINUE STATEMENT IN C LANGUAGE:-

- it is used to continue the execution of loop (while, do while and for). It is used with *if condition* within the loop.
- Syntax:-

jump-statement;

continue;

CONTINUE STATEMENT IN C LANGUAGE:-

```
#include<stdio.h>
void main ()
{
    int i = 0;
    while(i!=10)
    {
        printf("%d", i);
        continue;
        i++;
    }
}
```

GOTO STATEMENT

- The goto statement is a jump statement which is sometimes also referred to as unconditional jump statement. The goto statement can be used to jump from anywhere to anywhere within a function.
- Syntax1:

```
        goto label;
        ------
      -------
        label:
```

Syntax2:

```
 label:
        ------
      -------
     goto  label;
```

GOTO-EXAMPLE

```
#include <stdio.h>
void checkEvenOrNot(int num)
{
        if (num % 2 == 0)
                goto even;
        else
                goto odd;

even:
printf("%d is even", num);
        return;
odd:
printf("%d is odd", num);
}
```

```
// C program to check if a number is
// even or not using goto statement


int main()
 {
int num = 26;
checkEvenOrNot(num);
return 0;
}
```


THANK YOU